dbt Training — Exercises

What You're Building

Across Modules 01–07 you'll build a working dbt project from the staging layer up. Each session adds one layer to the same project. By the end of Module 07 you'll have a complete staging layer, a tested Silver fact table, and fully documented Silver dimensions.

The project you're handed at the start of Module 01:

```
models/
  staging/
    hubspot/
      sources.yml                        ← contacts + deals declared (no owners
yet)
      stg_hubspot__pipeline_stages.sql   ← pre-built with a deliberate bug
  silver/
    dim_pipeline.sql                     ← pre-built, no tests, no docs
    dim_patient.sql                      ← pre-built, no tests, no docs
    dim_doctor.sql                       ← pre-built, no tests, no docs
    fct_prescription.sql                 ← pre-built, no tests, no docs
```

The project you'll have after Module 07:

```
models/
  staging/
    hubspot/
      sources.yml                        ← extended: owners added with freshness
      stg_hubspot__contacts.sql          ← written in Module 03
      stg_hubspot__pipeline_stages.sql   ← fixed in Module 04
      stg_hubspot__deals.sql             ← written in Module 04
      stg_hubspot__owners.sql            ← written in Module 05
  silver/
    dim_pipeline.sql                     ← unchanged
    dim_patient.sql                      ← unchanged
    dim_doctor.sql                       ← unchanged
    fct_prescription.sql                 ← unchanged
    schema.yml                           ← created Module 06, extended Module 07

tests/
  assert_fct_prescription_no_zero_dosage.sql  ← written in Module 06
```

Reference Data

Sample data matching all exercises lives in `data/`:

```
data/
  bronze/
    hubspot_contacts.csv          ← source for stg_hubspot__contacts
    hubspot_deals.csv             ← source for stg_hubspot__deals
    hubspot_pipeline_stages.csv   ← source for stg_hubspot__pipeline_stages
    hubspot_owners.csv            ← source for stg_hubspot__owners
  silver/
    dim_pipeline.csv              ← expected output of dim_pipeline (SCD2 example)
    dim_patient.csv               ← expected output of dim_patient
    dim_doctor.csv                ← expected output of dim_doctor
    fct_prescription.csv          ← expected output of fct_prescription (clean
state)
```

Check these files before writing SQL. They'll show you the column names, data types, and expected values you're working toward.

Before You Start — Configure Your Profile

Copy `profiles.yml.example` to `~/.dbt/profiles.yml` and replace every placeholder with the correct value for your environment:

Key	Example
`account`	`xy12345.eu-west-1`
`user`	`jane@company.com`
`role`	`transformer_dev`
`warehouse`	`compute_wh_dev`
`database`	`analytics_dev`
`schema`	`dev_jane`

> **Never commit `profiles.yml`.** It's already in `.gitignore`. The `.example` file is the only profiles file that lives in the repo.

Once you've updated it, verify the connection before any exercise:

```
dbt debug
```

All checks should pass before you write any SQL.

Module 01 — Orientation (25 min)

Prerequisite: Module 01 theory block **Goal:** Navigate the project without guidance; understand what exists before you start building

Project state at start

You've been given a partially built dbt project. Nothing is yours yet — everything you see was scaffolded for you. Your first job is to understand what you've been handed.

Task 1 — Read `dbt_project.yml`

Open `dbt_project.yml`. Answer without Googling:

1. What is the project name?
2. What materialization is set for `staging` models?
3. What materialization is set for `silver` models?
4. What does the `+database` key under `silver` do?

Task 2 — Count and list Silver models

Navigate to `models/silver/`. List every `.sql` file. For each one, open it and find the first `{{ ref() }}` or `{{ source() }}` call.

Task 3 — Find the bug

Open `models/staging/hubspot/stg_hubspot__pipeline_stages.sql`. Something is wrong with this file. Can you spot it? Don't fix it yet — that's Module 04.

Task 4 — Run `dbt ls`

```
dbt ls
```

How many models are listed? Which layer has the most?

Module 02 — Local Setup

No coding exercise. The goal of this session is to get `dbt debug` to pass:

```
dbt debug
```

All checks must show `OK` before you proceed to Module 03.

Module 03 — Write Your First Staging Model (20 min)

Prerequisite: `dbt debug` passing; `profiles.yml` configured **Project state at start:** `sources.yml` exists with `contacts` declared. No staging SQL models yet.

Task 1 — Write `stg_hubspot__contacts.sql`

Create `models/staging/hubspot/stg_hubspot__contacts.sql`.

The Bronze source table (`BRONZE.HUBSPOT.contacts`) has these columns:

Column	Notes
`contact_id`	HubSpot contact ID
`email`	Contact email address
`pipeline_id`	The pipeline this contact belongs to
`_loaded_at`	Timestamp the row was loaded by Lambda

Write a staging model that:

- Is materialised as a `view`
- References the source via `{{ source('hubspot', 'contacts') }}`
- Selects all four columns, renaming `_loaded_at` → `loaded_at` (drop the leading underscore)

Task 2 — Compile and verify

```
dbt compile --select stg_hubspot__contacts
```

Open `target/compiled/analytics/models/staging/hubspot/stg_hubspot__contacts.sql`.

Verify:

- `{{ source('hubspot', 'contacts') }}` compiled to `BRONZE.HUBSPOT.contacts`
- `{{ config(materialized='view') }}` is gone — it compiled to a `CREATE OR REPLACE VIEW` wrapper

Task 3 — Predict before you check

Before running step 2, write down what you expect the compiled SQL to look like. Then compare your prediction to the actual output.

▶ Expected model

▶ Expected compiled SQL (dev target)

Project state at end of Module 03: 1 staging model. 0 staging models yet running in Snowflake.

Bonus — Variables and environment-aware filtering

dbt lets you define variables in `dbt_project.yml` and read them in any model with `{{ var('my_var') }}`. Combined with `{{ target.name }}`, this is a common pattern for limiting data volume in dev without changing your production query.

Your task (no code provided — use an AI assistant to help you work this out):

1. Add a variable called `limit_rows` with a default value to `dbt_project.yml` under the `vars:` key.
2. In `stg_hubspot__contacts.sql`, add a `WHERE` clause that only activates when the current target is `dev`. When active, it should use the `limit_rows` variable to restrict the number of rows returned.

3. Compile the model and inspect the output. Does the `WHERE` clause appear? What happens when the target is `prod`?
4. Override the variable from the command line with `--vars` and recompile. Confirm the compiled SQL changes.

Think about: why is this pattern useful in a team where Bronze tables have millions of rows?

Module 04 — Fix and Extend Staging (25 min)

Prerequisite: Module 03 complete **Project state at start:** `stg_hubspot__contacts.sql` exists. A broken `stg_hubspot__pipeline_stages.sql` is in the project.

Task 1 — Fix `stg_hubspot__pipeline_stages.sql`

Open the file. Find the materialisation problem. Fix it with a one-line change.

Then explain in one sentence why that materialisation was wrong for a staging model.

The Bronze source table (`BRONZE.HUBSPOT.pipeline_stages`) has these columns:

Column	Notes
`pipeline_stage_id`	HubSpot stage ID
`stage_name`	Human-readable stage label
`is_closed`	Whether this stage represents a closed deal
`pipeline_id`	The pipeline this stage belongs to
`_loaded_at`	Ingestion timestamp

After fixing the materialisation, make sure the model also selects all five columns (rename `_loaded_at` → `loaded_at`).

▶ The bug and fix

Task 2 — Write `stg_hubspot__deals.sql`

Create `models/staging/hubspot/stg_hubspot__deals.sql`.

The Bronze source table (`BRONZE.HUBSPOT.deals`) has these columns:

Column	Notes
`deal_id`	HubSpot deal ID
`deal_name`	Name of the deal
`pipeline_id`	Pipeline the deal belongs to
`close_date`	Expected close date
`_loaded_at`	Ingestion timestamp

Requirements:

- Materialise as view
- Source: `{{ source('hubspot', 'deals') }}`
- Select all five columns
- Rename `close_date` → `expected_close_date`
- Rename `_loaded_at` → `loaded_at`

▶ Expected model

Task 3 — Run all staging models

```
dbt run --select staging.*
```

Three models should show `OK`. Open Snowflake and verify that `stg_hubspot__contacts`, `stg_hubspot__pipeline_stages`, and `stg_hubspot__deals` exist as views in your dev schema (`TESTING__dev_yourname`).

Bonus — Spot the two bugs

The code below has two problems. Don't run it — identify both bugs by reading the code.

```
{{ config(
    materialized     = 'incremental',
    unique_key       = 'contact_key',
    on_schema_change = 'ignore'
) }}

SELECT contact_key, email, updated_at
FROM {{ ref('stg_hubspot__contacts') }}

{{ if is_incremental() }}
    WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{{ endif }}
```

▶ The two bugs

Project state at end of Module 04: 3 staging models running in Snowflake.

Module 05 — Add Sources and Complete the Staging Layer (25 min)

Prerequisite: Module 04 complete **Project state at start:** 3 staging models. `sources.yml` declares `contacts` and `deals`. `owners` is not yet declared.

Task 1 — Extend `sources.yml`

Open `models/staging/hubspot/sources.yml`. The file currently declares `contacts` and `deals` under the `hubspot` source.

Add the `owners` table to the same source block:

- Table name: `owners`
- `loaded_at_field`: `_loaded_at`
- Freshness warn threshold: 14 hours
- Freshness error threshold: 25 hours

▶ Expected entry to add

Task 2 — Write `stg_hubspot__owners.sql`

Create `models/staging/hubspot/stg_hubspot__owners.sql`.

The Bronze source table (`BRONZE.HUBSPOT.owners`) has these columns:

Column	Notes
`owner_id`	HubSpot owner ID
`first_name`	First name
`last_name`	Last name
`email`	Work email
`_loaded_at`	Ingestion timestamp

Requirements: view materialization, `{{ source('hubspot', 'owners') }}`, all five columns, rename `_loaded_at` → `loaded_at`.

▶ Expected model

Task 3 — Compile and verify

```
dbt compile --select stg_hubspot__owners
```

Confirm the compiled SQL references `BRONZE.HUBSPOT.owners` — not a hardcoded path you typed yourself.

Task 4 — Run and check freshness

```
dbt run --select staging.*
dbt source freshness
```

Four staging models should run `OK`. The freshness output will show `owners` as pass, warn, or error depending on when the source table was last loaded.

Project state at end of Module 05: 4 staging models. Full staging layer complete. All sources declared with freshness thresholds.

Module 06 — Test the Silver Layer (30 min)

Prerequisite: Module 05 complete **Project state at start:** Full staging layer. Silver models exist but `models/silver/schema.yml` does not yet exist.

Task 1 — Create `models/silver/schema.yml`

Create the file. Add a `models:` block for `fct_prescription`.

The model has these columns:

Column	Type	Constraint
`prescription_key`	Surrogate PK	unique + not_null (both `error`)
`patient_key`	FK → `dim_patient.patient_key`	not_null + relationships (both `error`)
`doctor_key`	FK → `dim_doctor.doctor_key`	not_null + relationships (both `error`)
`prescription_date`	Date	not_null (`error`)
`medication_type`	String	accepted_values: `tablet`, `liquid`, `injection`, `topical` (`error`)
`dosage_amount`	Number	not_null (`warn` — nulls are expected when dose not yet confirmed)
`notes`	String	no test required

Task 2 — Run the tests

```
dbt test --select fct_prescription
```

All tests pass on the clean dataset. Note how many tests ran and which model they're listed under.

Task 3 — Break a test deliberately

Manually insert a duplicate `prescription_key` in your dev schema:

```
INSERT INTO SILVER_DEV.TESTING__dev_yourname.fct_prescription
SELECT * FROM SILVER_DEV.TESTING__dev_yourname.fct_prescription
WHERE prescription_key = 'rx-001';
```

Run `dbt test --select fct_prescription` again. Read the failure output carefully:

- Which test failed?
- How many failure rows were found?
- Paste the test SQL from the output into Snowsight and run it manually.

Then restore clean data by re-running the model:

```
dbt run --select fct_prescription
```

Task 4 — Write a singular test

Create `tests/assert_fct_prescription_no_zero_dosage.sql`.

This test should return rows where `dosage_amount` is exactly `0`. A zero dosage is a data error — it means a prescription was recorded with zero quantity, which is impossible. A null dosage means "dose not yet confirmed" and is acceptable.

If the query returns any rows, the test fails.

▶ Expected schema.yml
▶ Expected singular test

Task 5 — Run `dbt build` on the Silver layer

```
dbt build --select silver.*
```

Watch the output order. Models run first; tests run immediately after each model in DAG order. `fct_prescription` won't build if `dim_patient` or `dim_doctor` fail their tests.

Project state at end of Module 06: `schema.yml` created. `fct_prescription` has 7 tests (6 error, 1 warn) plus the singular test.

Module 07 — Document the Silver Layer (20 min)

Prerequisite: Module 06 complete **Project state at start:** `schema.yml` exists with tests only. No model descriptions, no column descriptions, no grain statements.

Task 1 — Add documentation to `fct_prescription`

In `models/silver/schema.yml`, add a `description:` field to the `fct_prescription` model entry.

The description must include:

- A **grain statement**: what does one row in this model represent?
- At least one sentence about what the model is for

Then add `description:` fields to every column that already has a test.

Task 2 — Document `dim_pipeline`

Add a new entry to `schema.yml` for `dim_pipeline`. This model is SCD2 — one row per pipeline per validity period.

Columns to document:

Column	Type	Role
`pipeline_key`	Surrogate PK	Unique per pipeline version
`hubspot_pipeline_id`	String	Business key — stable across SCD2 versions
`pipeline_name`	String	Human-readable name
`is_active`	Boolean	Whether this pipeline is in use
`dbt_valid_from`	Timestamp	When this version became effective
`dbt_valid_to`	Timestamp	When superseded. NULL means current
`is_current`	Boolean	True if `dbt_valid_to IS NULL` — convenience flag

Requirements:

- Grain statement explaining the SCD2 pattern (see `data/silver/dim_pipeline.csv` for an example: `hs-pipeline-003` appears twice — once historical, once current)
- Description for every column
- Tests: `pipeline_key` (unique + not_null), `is_current` (not_null)

Task 3 — Generate and browse docs

```
dbt docs generate
dbt docs serve
```

In the browser:

1. Search for `fct_prescription`. Confirm your grain statement appears in the model description.
2. Search for `dim_pipeline`. Click into the lineage DAG. Find the upstream staging model.
3. Click on `dbt_valid_to`. Confirm your column description appears.
4. Return to the DAG. Trace the full lineage from Bronze source to `fct_prescription`.

▶ Expected schema.yml after both tasks

Project state at end of Module 07: Silver layer tested and documented. All CI requirements satisfied.

What You've Built

At the end of Module 07 you can:

- Write staging models using `{{ source() }}` and `{{ config() }}`

- Explain the difference between `{{ }}`, `{% %}`, and why staging is always a view
- Declare sources in `sources.yml` with freshness thresholds
- Run `dbt compile`, `dbt run`, `dbt test`, and `dbt build` and know when to use each
- Write a complete Silver test suite including generic tests, severity levels, and a singular test
- Write grain statements and column descriptions that satisfy CI requirements
- Serve and navigate the dbt docs site

Modules 08–16 — Tier 2 and Tier 3

Whats comes next?

Module	**Planned exercise**
08 — Advanced Materializations	Debug an incremental model with a late-arriving data bug; fix the lookback window
09 — Seeds and Variables	Add a status mapping seed; wire it into a Silver model via `ref()`
10 — Jinja and Macros	Write a `safe_divide` macro; use it in two models
11 — SCD2 and Snapshots	Compare native snapshot output against `scd2_merge` macro output
12 — Selectors	Write selector expressions to target specific layers; trace downstream impact
13 — CI/CD and Slim CI	Simulate a slim CI run using a saved `manifest.json`; observe which models are selected
14 — Advanced Testing	Add `store_failures: true` to a test; query the failure table in Snowsight
15 — Incremental Patterns	Reproduce the reopened-ticket bug; apply the fix
16 — Governance	Write a model contract for a Gold mart; break it deliberately; read the compile error